LEARN BEING FORWARD

Programming — JavaScript Programming

DOM manipulation & events

COMPLETE LEARNING GUIDE

What You'll Learn

JavaScript's most visible job in the browser is reading and changing what's actually on screen, and it does this through the Document Object Model (DOM) — a live, tree-shaped, in-memory representation of every element on a page that JavaScript can query and mutate directly. This sub-module covers the DOM as part of the language's practical toolkit: how the tree is structured, selecting and traversing elements, safely creating and modifying content, styling through classes, and the event system (listeners, the event object, and bubbling) that lets a page respond to user interaction. The emphasis here is on getting the core browser APIs right as a language feature, independent of any framework, since this is the exact layer every framework's rendering eventually compiles down to.

1. What Is the DOM? Nodes, Elements, and the Document Tree

When a browser loads an HTML page, it parses the markup into a tree of Node objects — Element nodes for tags, Text nodes for raw text content, Comment nodes for HTML comments — with the single document object as the tree's root. JavaScript doesn't just read this tree once; it's a live, mutable structure, which means changing it through JavaScript immediately and visibly updates what the user sees, with no separate 're-render' step required. Every element node also exposes useful metadata directly, like tagName, id, and classList, without needing to re-parse any HTML string.

2. Selecting Elements with querySelector and querySelectorAll

document.querySelector(selector) returns the first element matching any valid CSS selector, or null if nothing matches — that null case must always be checked before calling a method on the result, or the code throws. document.querySelectorAll(selector) returns every match as a static NodeList (a snapshot, not a live view), which supports .forEach() directly for iterating over all matches. These two methods, using ordinary CSS selector syntax, have effectively replaced the older getElementById/getElementsByClassName/getElementsByTagName for most new code, since one consistent selector syntax covers every case those older, narrower methods split apart.

```javascript
const title = document.querySelector("#page-title");
if (title) {
  title.textContent = "Welcome back!";
}

const allButtons = document.querySelectorAll(".btn");
allButtons.forEach((btn) => btn.classList.add("ready"));
```

3. Reading and Changing Content Safely

textContent reads or writes an element's content as plain text — it's always safe, since any string assigned to it is treated as literal text, never parsed as markup. innerHTML instead parses an assigned string as HTML and rebuilds the element's children from it, which is powerful for inserting structured markup but dangerous the moment any part of that string could come from untrusted input, since an embedded <script> or event-handler attribute could then execute arbitrary code (a cross-site scripting vulnerability). As a rule: use textContent whenever you're only inserting text, and reserve innerHTML for trusted, developer-controlled markup.

```javascript
const userBio = "<img src=x onerror=alert('hacked')>";
const bioEl = document.querySelector("#bio");

bioEl.textContent = userBio; // SAFE — rendered as literal visible text, not executed
// bioEl.innerHTML = userBio; // DANGEROUS if userBio is untrusted user input
```

4. Creating and Removing Elements

document.createElement(tagName) builds a brand-new, detached element that exists only in memory until it's inserted into the tree with appendChild(), append(), prepend(), or insertBefore(). Removing an element from the page is done with element.remove() directly on the node itself (a more recent, simpler API than the older parent.removeChild(element) pattern, though both still work). Building several new elements inside a DocumentFragment before appending them all in one operation avoids triggering a separate visual update for every single insertion when adding many elements at once.

```javascript
const list = document.querySelector("#fruit-list");
const fruits = ["Apple", "Mango", "Banana"];

const fragment = document.createDocumentFragment();
fruits.forEach((fruit) => {
  const li = document.createElement("li");
  li.textContent = fruit;
  fragment.appendChild(li);
});
list.appendChild(fragment); // one single DOM update for all three items
```

5. Styling via classList and the style Property

classList (.add(), .remove(), .toggle(), .contains()) is the preferred way to change an element's appearance, since it works together with CSS rules already defined in a stylesheet rather than hardcoding visual values directly in JavaScript. The style property sets inline CSS directly on an element (element.style.display = 'none') and is appropriate specifically for values computed at runtime that genuinely can't be expressed as a predefined, toggleable class, such as a dynamically calculated pixel offset.

```javascript
const panel = document.querySelector("#panel");

panel.classList.toggle("open"); // flips current state, doesn't force a fixed outcome
console.log(panel.classList.contains("open")); // true or false, depending on prior state
```

6. Handling Events with addEventListener

element.addEventListener(type, handler) attaches a listener function to run when the given event fires, and — unlike the older element.onclick = handler pattern, which allows only one handler per element and silently overwrites any previous one — addEventListener allows any number of independent listeners on the same element and event type. The handler function receives an event object with useful properties: event.target (the exact element the event originated on) and methods like event.preventDefault() (cancels a default browser action, like a form's full-page submission or a link's navigation).

```javascript
const form = document.querySelector("#login-form");

form.addEventListener("submit", (event) => {
  event.preventDefault(); // stop the default full-page reload on submit
  const formData = new FormData(form);
  console.log(Object.fromEntries(formData));
});
```

7. Event Bubbling and Simple Delegation

Most DOM events bubble: after firing on the exact element the interaction happened on, the same event fires again on each ancestor element, all the way up to document. This makes event delegation possible — instead of attaching one listener to every item inside a list (wasteful, and automatically broken for any item added to the list later), attach a single listener to a stable parent container and inspect event.target inside it to figure out which specific child was actually clicked. closest(selector) is the standard tool for finding the nearest matching ancestor (including the element itself) from inside a delegated handler.

```javascript
const list = document.querySelector("#todo-list");

list.addEventListener("click", (event) => {
  const item = event.target.closest("li"); // works for existing AND future <li> elements
  if (!item) return;
  item.classList.toggle("done");
});
```

8. Working with Forms and Basic Validation

Built-in HTML validation attributes (required, minlength, pattern, type="email") catch many common input errors automatically, before JavaScript even runs, and checkValidity() can be called on a form element to check this programmatically. For validation logic HTML attributes can't express — comparing two password fields, checking a value against data fetched from a server — JavaScript reads each field's .value, runs the custom check in the submit handler, and calls event.preventDefault() to block submission until the problem is fixed, typically alongside setting a visible error message element's text for the user.

```javascript
form.addEventListener("submit", (event) => {
  const password = form.querySelector("#password").value;
  const confirm = form.querySelector("#confirm-password").value;

  if (password !== confirm) {
    event.preventDefault();
    document.querySelector("#error").textContent = "Passwords do not match.";
  }
});
```

Common Pitfalls

- Calling a method directly on the result of querySelector without checking for null first, crashing when the selector matches nothing on the page.
- Setting innerHTML with any string that includes user-provided content, opening a cross-site scripting (XSS) vulnerability instead of using textContent.

- Forgetting event.preventDefault() in a form submit handler, causing an unwanted full-page reload that discards any custom validation logic.
- Attaching an individual event listener to every item in a dynamic list instead of using delegation, so items added later never receive the handler.
- Using element.onclick = handler, which silently overwrites any previously assigned handler instead of adding an additional one like addEventListener does.
- Assuming classList.toggle() always adds a class — it flips the current state, which surprises code that expected a guaranteed, fixed outcome.
- Treating querySelectorAll's NodeList as fully array-like for every array method — it supports forEach() but not map()/filter() directly without converting it first with Array.from().

Key Takeaways

- The DOM is a live, mutable tree of nodes — changing it through JavaScript immediately updates what's rendered on screen.
- querySelector/querySelectorAll with ordinary CSS selectors should be the default way to find elements in new code.
- textContent is always safe for inserting text; innerHTML is only safe with fully trusted, developer-controlled markup.
- addEventListener supports multiple independent listeners per element and event type; onclick = handler allows only one and overwrites silently.
- Event bubbling is what makes delegation possible — one listener on a stable parent can handle clicks on children that don't exist yet.
- Built-in HTML validation attributes handle many basic cases; JavaScript fills in whatever validation logic those attributes can't express.

Learn More

MDN — Introduction to the DOM
https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction

MDN — EventTarget.addEventListener()
https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener

MDN — Element: closest() method
https://developer.mozilla.org/en-US/docs/Web/API/Element/closest